Vision Lego

Developer Blueprint — Your personal reference. Not for Claude Code. Updated May 2026 to reflect Wompi payments, the completed feature set, the image pipeline, and the current droplet configuration.

The big picture

You are building a fullstack web app called **Vision Lego** — an online eyewear store with two sides:

- A public store (in Spanish) where customers browse and buy sunglasses and optical frames.
- A private dashboard at **/negocio** where the store owner manages products, inventory, orders, expenses, receivables, analytics, and shipping methods.

The whole thing lives in one SvelteKit project on your laptop. Claude Code writes the code, you review and guide it. When you are happy, you push to **main** on GitHub and GitHub Actions deploys it to the DigitalOcean droplet automatically.

Your tools and what they do

Tool	What it is	Why you're using it
SvelteKit	The web framework	Handles frontend and backend in one project (SSR by default)
TypeScript	Typed JavaScript	Catches errors before they happen
pnpm	Package manager	Faster than npm; the only package manager used in this project
Drizzle	Database ORM	Talk to PostgreSQL using TypeScript instead of raw SQL
PostgreSQL	Database	Stores all your data (products, orders, users, expenses, etc.)
Better Auth	Auth library	Email + password login, sessions, role-based access
Tailwind CSS	Utility framework	Styling without writing custom CSS files; class-based dark mode
sharp	Image processing	Server-side resize and WebP encoding for every product image
cropperjs v1.6	Client-side cropper	Square crop + zoom modal before each upload in /negocio
Wompi	Payment provider	Colombian card + PSE payments; lower fees than MercadoPago
Vitest	Unit testing	Tests individual functions and components
Playwright	E2E testing	Tests the app like a real user in a browser
PM2	Process manager	Keeps the Node.js server running 24/7 on the droplet
Nginx	Reverse proxy	Receives port-80 traffic and forwards it to your app on port 3000
GitHub Actions	CI/CD	Automatically deploys when you push to main

Heads up: payments switched from MercadoPago to Wompi by client decision (lower transaction fees in Colombia). When you start the integration you will rename the schema fields **mpPaymentId** → **wompiPaymentId** and **mpStatus** → **wompiStatus** via a Drizzle migration.

How the code is organized

```
vision_lego/
├── src/
│   ├── lib/
│   │   ├── server/
│   │   │   ├── db/
│   │   │   │   ├── schema.ts     ← All database tables
│   │   │   │   └── index.ts      ← DB connection
│   │   │   ├── auth.ts           ← Better Auth config
│   │   │   ├── cartCookie.ts     ← Guest cart cookie helpers
│   │   │   └── imageProcessing.ts ← sharp pipeline (thumb / medium / large + original)
│   │   ├── components/
│   │   │   ├── ui/               ← ThemeToggle, ProductImage, ImageCropper, etc.
│   │   │   ├── store/            ← Customer-facing components
│   │   │   └── negocio/          ← Owner dashboard components
│   │   ├── theme.svelte.ts       ← Dark/light $state store
│   │   └── utils/
│   │       └── currency.ts       ← formatCOP()
│   ├── hooks.server.ts           ← Protects /negocio/** routes
│   └── routes/
│       ├── (store)/              ← Public store pages
│       ├── (auth)/               ← /iniciar-sesion, /registrarse
│       └── negocio/              ← Owner dashboard (role: owner | staff)
├── static/uploads/products/      ← In .gitignore; images live on the server
├── docs/IMAGE_STANDARDS.md       ← Image pipeline spec
├── guia-fotos-productos.txt      ← Plain-Spanish photo guide for the owner
├── .env                          ← NEVER commit
├── drizzle.config.ts
├── ecosystem.config.cjs          ← PM2 config (generated)
├── nginx.conf                    ← Nginx HTTP reverse proxy (generated)
└── .github/workflows/deploy.yml ← GitHub Actions deploy (generated)
```

Key rule: anything in **src/lib/server/** never reaches the browser. DB queries, auth, secret keys, and the image pipeline all live there.

The database

PostgreSQL stores everything. You talk to it through Drizzle, so you write TypeScript instead of SQL. Schema lives at **src/lib/server/db/schema.ts**.

Your tables at a glance

Table	What it stores
users	Everyone with an account (customer, owner, staff)
sessions	Active login sessions (Better Auth)
accounts	Auth providers + password hashes (Better Auth)
verifications	Email verification tokens (Better Auth)
brands	Eyewear brands
products	Sunglasses and optical frames
productVariants	Each color / size / material combo

Table	What it stores
productImages	Photos per product, optionally tagged to a variant
cartItems	Logged-in user's cart (guests use a cookie instead)
wishlistItems	Items a customer saved for later
orders	Every purchase (userId is nullable — guests allowed)
orderItems	Line items inside each order (snapshots of name + price)
expenses	Business expenses the owner logs
receivables	Outstanding debts and partial payments
shippingMethods	Delivery options and their COP cost

Migrations

When you change **schema.ts**, sync the change to the real database:

```
pnpm drizzle-kit generate   # creates a migration SQL file
pnpm drizzle-kit migrate    # applies it to the database
```

You never write SQL by hand.

Authentication and roles

Everyone logs in at **/iniciar-sesion**. Better Auth checks the role and redirects accordingly.

Role	Where they go after login	What they can do
customer	/	Browse, add to cart, check out, manage their profile
owner	/negocio	Full business dashboard
staff	/negocio	Same as owner (role exists in schema, no UI yet)

The owner account is created manually via SQL UPDATE. There is no public admin registration. Guests can browse and check out without an account — their cart lives in a 30-day cookie (**guest_cart**) and orders are created with **userId = null**; the confirmation page is **/pedido/[id]**.

Two Better Auth gotchas already fixed:

- The Drizzle adapter needs **usePlural: true** and the schema passed explicitly.
- **role** must be declared as an **additionalField** in the auth config.
- Right after **signInEmail** in a SvelteKit form action, **getSession()** returns null — query the role directly from the DB to do the redirect.

Environment variables

Secret values your app needs to run. They live in **.env** at the project root. **Never** commit **.env** to GitHub (it is in **.gitignore**). **CLAUDE.md** is fine to commit — it has no secrets.

```
# Local development
DATABASE_URL=postgresql://mikey:your_strong_password@localhost:5432/vision_lego
BETTER_AUTH_SECRET=generate_with_openssl_rand_base64_32
BETTER_AUTH_URL=http://localhost:5173

# Production values live as GitHub Actions secrets, not in any file.
```

Wompi merchant credentials (public key + private key) are added when payment integration begins. Do not start the Wompi step until the owner provides them.

The server (DigitalOcean Droplet)

What	Details
Plan	Basic Intel — 1 vCPU, 1 GB RAM, 35 GB disk (~$8/mo)
OS	Ubuntu 24.04.4 LTS
Node.js (local)	v24.15.0
Node.js (droplet)	v24.14.1 (Node 24 is LTS since Oct 2025)
PostgreSQL	Installed and configured. DB: vision_lego, owner: mikey
Nginx	Latest — listens on port 80, reverse-proxies to localhost:3000
PM2	Installed globally; keeps build/index.js alive 24/7
Users	root and mikey. Always use mikey. Never SSH as root.
sudo for mikey	Passwordless, scoped only to Nginx + PM2 commands
SSH key (local)	~/.ssh/Digital_Ocean_Key
HTTPS	Not yet — no domain. App is reached by IP, HTTP only.

How traffic flows

```
Browser  →  Nginx (port 80)  →  your app (port 3000)  →  PostgreSQL
```

Nginx is the public face. Your Node.js app listens on the internal port 3000 and is never exposed directly.

One-time server setup

Most of this is already done on the current droplet. Listed here in order so you can rebuild from scratch if you ever move to a new droplet.

1. PostgreSQL (done)

```
sudo -u postgres psql
-- inside psql --
CREATE USER mikey WITH PASSWORD 'your_strong_password';
CREATE DATABASE vision_lego OWNER mikey;
GRANT ALL PRIVILEGES ON DATABASE vision_lego TO mikey;
\q
```

2. PM2 (done)

PM2 is installed globally. After your first successful deploy:

```
pm2 startup   # copy and run the command it prints (sudo systemctl ...)
pm2 save      # saves the process list so it survives reboots
```

3. Let Nginx read your uploads directory (do this once)

Ubuntu 24 creates **/home/mikey** with mode 0750, so the **www-data** user that runs Nginx cannot traverse it to reach **static/uploads/**. Without this step every uploaded product image returns HTTP 403 in production:

```
sudo chmod o+x /home/mikey
```

4. Nginx config (HTTP only, generated by Claude Code)

```
sudo cp nginx.conf /etc/nginx/sites-available/vision_lego
sudo ln -s /etc/nginx/sites-available/vision_lego /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default   # disable the default site
sudo nginx -t && sudo systemctl reload nginx
```

One-time GitHub setup (already done)

All these secrets are added under **Settings** → **Secrets and variables** → **Actions**:

Secret name	Value
DROPLET_HOST	Droplet's public IP address
DROPLET_USER	mikey (never root)
DROPLET_SSH_KEY	Contents of ~/.ssh/Digital_Ocean_Key (the private key)
DATABASE_URL	postgresql://mikey:password@localhost:5432/vision_lego
BETTER_AUTH_SECRET	Same long random string as the local .env
BETTER_AUTH_URL	http://YOUR_DROPLET_IP today — https://yourdomain.com once HTTPS is set up

The development workflow (day to day)

```
1. cd ~/Git_Repos/vision_lego
2. pnpm dev                  # app runs at http://localhost:5173
3. Ask Claude Code to build a feature; review the diff
4. Test locally in the browser (mobile + desktop, light + dark)
5. git add . && git commit -m "feat: short message"
6. git push origin main
   → GitHub Actions deploys to the droplet (~1–2 min)
   → Site updates live
```

You never copy files to the server by hand or restart PM2 yourself — Actions does it.

What happens after git push

```
You push to main on GitHub
■
GitHub Actions starts (runs on GitHub's servers, not yours)
■
Runs pnpm install + pnpm build locally on the runner
■
SSHes into your droplet as `mikey`
■
Copies build/, package.json, pnpm-lock.yaml to the droplet
■
Runs `pnpm install --prod` on the droplet (production deps only)
■
Runs `pm2 restart vision_lego`  (or `pm2 start ecosystem.config.cjs` on first deploy)
■
Your app is live with the new version  ✓
```

Product images — what already works

Every product image goes through a real pipeline. The droplet is small (1 GB RAM, single-core) and customers are on Colombian mobile networks, so serving raw iPhone photos would be unusable. Here is the flow:

- Owner picks file(s) in **/negocio/productos/[id]**.
- Each file opens in a square **cropperjs** modal (1:1 locked, max zoom 100%, output 1600×1600 WebP @ 0.92).
- Client-side check via **createImageBitmap**: reject anything under 1200×1200 px with a Spanish error.
- Cropped Blob is appended to FormData and sent to the **uploadImages** action.
- **processProductImage()** uses **sharp** to generate four files per image: -thumb.webp (400×400 q80), -medium.webp (800×800 q85), -large.webp (1600×1600 q85), and the unmodified -original (kept for future reprocessing).
- **productImages.url** stores the basePath without an extension; the **ProductImage** component renders a full srcset/sizes <picture> element.

Full spec lives in **docs/IMAGE_STANDARDS.md**. Plain-Spanish photo guide for the owner is **guia-fotos-productos.txt**.

Completed features (steps 1–15)

All non-payment functionality is built and working. The implementation order from CLAUDE.md is complete through step 15:

- **Schema + migrations** — full Drizzle schema, all migrations applied.
- **Better Auth** — email + password, role-based redirect, route protection in **src/hooks.server.ts**.
- **Product CRUD** — create / edit / delete, activate, feature toggle, per-variant image assignment, inline variant editing.
- **Image upload** — multi-file, square cropper modal, sharp pipeline, ProductImage component everywhere.
- **Inventory** — inline stock edit, low-stock alerts (≤ 3 highlighted red), low-stock filter.
- **Public catalog** — SSR URL-param filters (category, brand, gender, color, material, size, price), sort options.
- **Product detail** — variant selector, swipeable image gallery, stock indicator, add to cart, add to wishlist.
- **Cart** — DB-persisted for logged-in users; cookie-based for guests, with proper optimistic-update + explicit formData.set() fix for Svelte 5 timing.
- **Checkout** — shipping form, shipping method selector, order created with status **pending**; **guests welcome** (userId null, confirmation at /pedido/[id]).
- **Order management** — /negocio/pedidos list + detail + status change.
- **Expenses, receivables, analytics, shipping methods** — all built in /negocio.
- **Customer account** — /cuenta with order history, profile editor, wishlist.
- **Dark / light mode toggle** — system preference + localStorage, anti-flash inline script, all 27 components have dark: variants.
- **Mobile-first responsive design** — 44px tap targets, Tailwind responsive prefixes throughout.
- **Deployment artifacts** — deploy.yml, ecosystem.config.cjs, nginx.conf already generated and on the server.

What is still pending

Step 16 — Wompi payment integration

- Replaces MercadoPago by client decision (lower fees in Colombia).
- Before writing any payment code, generate a Drizzle migration that renames **orders.mpPaymentId** → **wompiPaymentId** and **orders.mpStatus** → **wompiStatus**.
- Integration is via Wompi's REST API (server-side order creation + signature) or their JS widget — docs at **https://docs.wompi.co**.
- Wire a **/api/wompi-webhook** endpoint to receive payment event notifications; on success, set order status to **paid** and store the IDs.
- Do **not** start this step until the owner gives you their Wompi merchant credentials (public + private keys).
- Test end-to-end in the Wompi sandbox before going live.

First production deploy

- Everything needed is generated — push to main and let Actions do the rest.
- After the very first successful deploy, run **pm2 startup** + **pm2 save** on the droplet so the process survives reboots.
- Verify image uploads serve correctly — if you get 403s, you forgot the **sudo chmod o+x /home/mikey** step.

Domain + HTTPS (later)

- Buy a domain, point its A record at the droplet IP.
- Run Certbot (Let's Encrypt) for a free TLS certificate.
- Update the **BETTER_AUTH_URL** Actions secret to **https://yourdomain.com**.
- Update nginx.conf to redirect HTTP → HTTPS and terminate SSL.

Optional polish

- A **site_settings** table so the owner can edit hero text, trust signals, accent color from /negocio.
- Better Auth email verification (currently the field exists but is not enforced).
- Move to AVIF when browser support is fully there — the originals are kept so a reprocessing script is enough.

Quick reference — useful commands

```
# Local development
pnpm dev                    # start dev server at localhost:5173
pnpm build                  # build for production (output: build/)
pnpm preview                # preview the production build locally

# Database
pnpm drizzle-kit generate   # generate migration after schema changes
pnpm drizzle-kit migrate    # apply migrations to the database
pnpm drizzle-kit studio     # visual DB browser at localhost:4983

# Testing
pnpm test                   # Vitest unit tests
pnpm test:e2e               # Playwright end-to-end tests
```

```
# Deploy
git push origin main        # triggers automatic deployment

# SSH into droplet (always as mikey, never root)
ssh -i ~/.ssh/Digital_Ocean_Key mikey@YOUR_DROPLET_IP

# On the droplet
pm2 status
pm2 logs vision_lego        # live logs (Ctrl+C to exit)
pm2 restart vision_lego     # manual restart if needed
sudo systemctl reload nginx # after editing nginx config
```

Conventions you should not break

- Never SSH as **root** on the droplet — always **mikey**.
- Never commit **.env** to Git. **CLAUDE.md** on the other hand is fine to commit.
- **static/uploads/** is in .gitignore — images live on the server, not in Git.
- Currency: always render prices with **formatCOP()** from **src/lib/utils/currency.ts**. No decimals.
- Source code, variable names, comments, filenames: **English**. UI strings and URL slugs: **Spanish**.
- Use **crypto.randomUUID()** for every entity ID.
- Svelte 5 runes only — never legacy **$:** or **let** reactivity.
- Do not add new npm/pnpm dependencies without flagging it first.
- No popups, no email-capture interrupts, no ad scripts, no auto-playing carousels.